AI-Generated Smart Contract Security Audit Report

By Savant.Chat

Aave V3.7 — Security Audit Report

Table of contents

1. Info 3
2. Scope 3
3. Limitations 3
3.1. AI Powered Audit 3
3.2. Human Triage Scope 3
3.3. Disclaimer & Non-Warranty 4
4. Vulnerability Statistics 4
5. Low Issues 4
5.1. Rounding divergence in hasNoCollateralLeft prevents bad debt realization for multi-borrow positions 4
5.2. Strict Collateral Validation Blocks Bad Debt Realization for Fully Consumed Positions 6
6. Conclusion 8

1. Info

This report was generated by an AI smart contract auditor agent SavantChat and triaged by smart contract auditor Igor Gulamov.

Repository: aave-dao/aave-v3-origin

2. Scope

The audit covered the following in-scope files:

File
src/contracts/interfaces/IPool.sol
src/contracts/interfaces/IPoolConfigurator.sol
src/contracts/interfaces/IPoolDataProvider.sol
src/contracts/protocol/libraries/configuration/ReserveConfiguration.sol
src/contracts/protocol/libraries/configuration/UserConfiguration.sol
src/contracts/protocol/libraries/helpers/Errors.sol
src/contracts/protocol/libraries/logic/BorrowLogic.sol
src/contracts/protocol/libraries/logic/ConfiguratorLogic.sol
src/contracts/protocol/libraries/logic/GenericLogic.sol
src/contracts/protocol/libraries/logic/LiquidationLogic.sol
src/contracts/protocol/libraries/logic/PoolLogic.sol
src/contracts/protocol/libraries/logic/SupplyLogic.sol
src/contracts/protocol/libraries/logic/ValidationLogic.sol
src/contracts/protocol/libraries/math/PercentageMath.sol
src/contracts/protocol/libraries/types/DataTypes.sol
src/contracts/protocol/pool/Pool.sol
src/contracts/protocol/pool/PoolConfigurator.sol

3. Limitations

3.1. AI Powered Audit

This AI-powered audit by Savant.Chat is based on the smart contract code at the time of evaluation; any subsequent changes may require re-auditing. Due to AI limitations, such as potential hallucination, the audit may not detect all issues, and findings—especially fix recommendations—should be reviewed by a security specialist. Users are advised to implement additional checks like manual reviews and regular re-audits. The AI auditor is provided "as is," and neither Savant.Chat, nor its developers, nor its providers, shall be liable for any direct, indirect, incidental, special, exemplary, or consequential damages arising from its use or reliance on its results. Users must independently verify findings and assume all risks, as the AI may produce incorrect information.

3.2. Human Triage Scope

The human reviewer's involvement was strictly limited to the triage of findings generated by the AI agent and the validation of specific fixes implemented by the team. The reviewer did not conduct an independent manual audit or line-by-line code inspection. Consequently, any vulnerabilities or logical errors not detected by the AI agent (false negatives) are outside the scope of this human review. The verification provided confirms only that the reported AI findings were addressed; it does not certify the overall security of the codebase.

3.3. Disclaimer & Non-Warranty

This report represents the auditor's opinion based solely on the scope defined above. It does not guarantee that the code is bug-free or invulnerable to exploits. The triage process is limited to validating specific AI outputs and does not constitute a comprehensive security audit. The auditor assumes no liability for any direct or indirect losses, including loss of funds, arising from the use of this code. This report is not financial or investment advice.

4. Vulnerability Statistics

Severity	Count
Low	2

5. Low Issues

5.1. Rounding divergence in hasNoCollateralLeft prevents bad debt realization for multi-borrow positions

- aave-dao/aave-v3-origin/dd107.../src/contracts/protocol/libraries/logic/LiquidationLogic.sol#L155-L423

File: src/contracts/protocol/libraries/logic/LiquidationLogic.sol
Severity: Low
Status: Fixed

src/contracts/protocol/libraries/logic/LiquidationLogic.sol#L155-L423

```
function executeLiquidationCall(
  // mappings and params abbreviated
  DataTypes.ExecuteLiquidationCallParams memory params
) external {
  LiquidationCallLocalVars memory vars;
  DataTypes.UserConfigurationMap storage borrowerConfig = usersConfig[params.borrower];
  // reserve caches, updateState, calculateUserAccountData -> vars.totalCollateralInBaseCurrency
  vars.borrowerCollateralBalance = IAToken(vars.collateralReserveCache.aTokenAddress)
    .scaledBalanceOf(params.borrower)
    .getATokenBalance(vars.collateralReserveCache.nextLiquidityIndex);
  vars.borrowerReserveDebt = IVariableDebtToken(vars.debtReserveCache.variableDebtTokenAddress)
    .scaledBalanceOf(params.borrower)
    .getVTokenBalance(vars.debtReserveCache.nextVariableBorrowIndex);
  // validation, bonus, prices, units, close factor skipped
  vars.actualDebtToLiquidate = vars.borrowerReserveDebt; // full debt, bypass dust
  (vars.actualCollateralToLiquidate,
   vars.liquidationProtocolFeeAmount,
   vars.collateralToLiquidateInBaseCurrency) = _calculateAvailableCollateralToLiquidate(
    vars.collateralReserveCache.reserveConfiguration,
    vars.collateralAssetPrice, vars.collateralAssetUnit,
    vars.debtAssetPrice, vars.debtAssetUnit,
    vars.actualDebtToLiquidate, vars.borrowerCollateralBalance, vars.liquidationBonus
  ); // floor rounding: collateralAmount = borrowerCollateralBalance - 1
  bool hasNoCollateralLeft = vars.totalCollateralInBaseCurrency == vars.collateralToLiquidateInBaseCurrency; //
false
  _burnDebtTokens(vars.debtReserveCache, /*debtReserve*/, borrowerConfig, params.borrower, params.debtAsset,
    vars.borrowerReserveDebt, vars.actualDebtToLiquidate, hasNoCollateralLeft, /*strategy*/);
  // assume receiveAToken=true
  IAToken(vars.collateralReserveCache.aTokenAddress).transferOnLiquidation(
    params.borrower, params.liquidator, vars.actualCollateralToLiquidate,

vars.actualCollateralToLiquidate.getATokenTransferScaledAmount(vars.collateralReserveCache.nextLiquidityIndex),
    vars.collateralReserveCache.nextLiquidityIndex
  );
  // protocol fee: independent rayDivCeil rounds up
  if (vars.liquidationProtocolFeeAmount != 0) {
    uint256 scaledFee =
vars.liquidationProtocolFeeAmount.getATokenTransferScaledAmount(vars.collateralReserveCache.nextLiquidityIndex);
    uint256 borrowerScaled = IAToken(vars.collateralReserveCache.aTokenAddress).scaledBalanceOf(params.borrower);
    if (scaledFee > borrowerScaled) scaledFee = borrowerScaled;
    IAToken(vars.collateralReserveCache.aTokenAddress).transferOnLiquidation(
      params.borrower, /*treasury*/, vars.liquidationProtocolFeeAmount, scaledFee,
vars.collateralReserveCache.nextLiquidityIndex
    );
  }
  if (IAToken(vars.collateralReserveCache.aTokenAddress).scaledBalanceOf(params.borrower) == 0) {
    borrowerConfig.setUsingAsCollateral(collateralReserve.id, params.collateralAsset, params.borrower, false);
  }
  if (hasNoCollateralLeft && borrowerConfig.isBorrowingAny()) { // skipped!
    _burnBadDebt(/*...*/);
  }
}
```

Synopsis

A rounding-induced false negative in the `hasNoCollateralLeft` equality check prevents the `_burnBadDebt` path from triggering. This leaves stranded bad debt in the protocol when a user's final collateral is fully consumed but their remaining debt in other reserves is not cleared.

Technical Details

In `LiquidationLogic.executeLiquidationCall`, the `hasNoCollateralLeft` boolean is evaluated early during its computation by comparing `vars.totalCollateralInBaseCurrency` with `vars.collateralToLiquidateInBaseCurrency` to determine if the user's bad debt should be realized across all reserves. It compares `totalCollateralInBaseCurrency` (derived from the full `borrowerCollateralBalance`) with `collateralToLiquidateInBaseCurrency` (derived from `collateralAmount`).

If a liquidator fully repays the debt in the current reserve (`actualDebtToLiquidate == borrowerReserveDebt`), the dust check at the `if` condition enforcing the `MustNotLeaveDust` requirement (checking partial debt and collateral liquidation) is bypassed. Due to floor rounding in `percentMulFloor` when calculating `maxCollateralToLiquidate`, `collateralAmount` can evaluate to exactly 1 wei less than `borrowerCollateralBalance`. This causes `hasNoCollateralLeft` to evaluate to `false`.

However, when the unscaled `collateralAmount` is split into the liquidator share and protocol fee, and converted to scaled amounts, the independent `rayDivCeil` operations can round up such that the total scaled amount subtracted exactly equals the user's entire `scaledBalanceOf`.

As a result, the user's scaled balance becomes exactly `0`, and their collateral flag is successfully cleared at the post-transfer `scaledBalanceOf(params.borrower) == 0` check. When execution reaches the `if (hasNoCollateralLeft && borrowerConfig.isBorrowingAny())` condition

guarding `_burnBadDebt`, `hasNoCollateralLeft` is `false`, so the `_burnBadDebt` call is skipped. The user is left with `0` collateral (flag cleared) but still has debt in other reserves. Because their collateral flag is cleared, they can no longer be liquidated, leaving the remaining debt stranded forever without recording a deficit.

Verification Checklist

1. Verify that `hasNoCollateralLeft` is evaluated using unscaled base currency amounts before the burn/transfer operations.
2. Verify that `collateralAmount` can be slightly less than `borrowerCollateralBalance` due to floor rounding in `percentMulFloor`.
3. Verify that the dust check (`MustNotLeaveDust`) is bypassed when `actualDebtToLiquidate == borrowerReserveDebt`.
4. Verify that independent `rayDivCeil` operations during scaled amount conversion can sum up to the total `scaledBalanceOf` even when `collateralAmount < borrowerCollateralBalance`.
5. Verify that `_burnBadDebt` relies on the potentially false `hasNoCollateralLeft` boolean instead of the actual post-transfer collateral bitmap state.

Assumptions and Preconditions

1. The user has collateral in one reserve and debt in at least two reserves.
2. The user's position is liquidatable (Health Factor < 1).
3. The liquidator fully repays the debt in the target reserve (`actualDebtToLiquidate == borrowerReserveDebt`).
4. The required collateral for the repaid debt evaluates to exactly `borrowerCollateralBalance - 1` due to floor rounding.
5. The independent `rayDivCeil` conversions for the liquidator share and protocol fee round up to equal the user's total `scaledBalanceOf`.

Proof of Concept

1. A user supplies 1000 USDC as collateral.
2. The user borrows 500 WETH and 5000 DAI.
3. Market movements make the position liquidatable.
4. A liquidator calls `liquidationCall` to repay WETH, setting `debtToCover` to fully repay the 500 WETH debt (`actualDebtToLiquidate == borrowerReserveDebt`).
5. Due to floor rounding in `percentMulFloor` when applying the liquidation bonus, `maxCollateralToLiquidate` evaluates to exactly 999 USDC (`borrowerCollateralBalance - 1`).
6. `collateralAmount` is set to 999 USDC. `hasNoCollateralLeft` evaluates to `false` because `collateralToLiquidateInBaseCurrency` (based on 999 USDC) != `totalCollateralInBaseCurrency` (based on 1000 USDC).
7. The dust check is bypassed because `actualDebtToLiquidate == borrowerReserveDebt`.
8. The 999 USDC is split into the liquidator share and protocol fee. When converting these unscaled amounts to scaled amounts using `rayDivCeil`, the sum of the scaled amounts rounds up to exactly equal the user's total `scaledBalanceOf`.
9. The user's `scaledBalanceOf` becomes exactly `0`, and the USDC collateral flag is cleared.
10. At the `if (hasNoCollateralLeft && borrowerConfig.isBorrowingAny())` condition guarding `_burnBadDebt`, `hasNoCollateralLeft` is `false`, so `_burnBadDebt` is skipped.
11. The user now has `0` collateral (flag cleared) but still has 5000 DAI debt. This debt is stranded forever, and the deficit is never recorded.

Remediation

Replace the `hasNoCollateralLeft` check at the `if (hasNoCollateralLeft && borrowerConfig.isBorrowingAny())` condition guarding the `_burnBadDebt` call with `!borrowerConfig.isUsingAsCollateralAny()`. This directly queries the post-transfer state of the user's collateral bitmap, ensuring that bad debt is correctly realized whenever the user's last collateral is fully consumed, regardless of any rounding divergence in the base currency calculations.

```
if (!borrowerConfig.isUsingAsCollateralAny() && borrowerConfig.isBorrowingAny()) {
```

Status

Fixed at 75145ad021c123a14d9190c67bd30e6867e22cb7

5.2. Strict Collateral Validation Blocks Bad Debt Realization for Fully Consumed Positions

- aave-dao/aave-v3-origin/dd107.../src/contracts/protocol/libraries/logic/ValidationLogic.sol#L246-L292

File: src/contracts/protocol/libraries/logic/ValidationLogic.sol

Severity: Low

Status: Fixed

src/contracts/protocol/libraries/logic/ValidationLogic.sol#L246-L292

```
function validateLiquidationCall(
    DataTypes.UserConfigurationMap storage borrowerConfig,
    DataTypes.ReserveData storage collateralReserve,
    DataTypes.ReserveData storage debtReserve,
    DataTypes.ValidateLiquidationCallParams memory params
  ) internal view {
    // Omitted: LocalVars declaration, self-liquidation check, reserve active/paused flags,
    // liquidation grace period checks

    require(
      params.healthFactor < HEALTH_FACTOR_LIQUIDATION_THRESHOLD,
      Errors.HealthFactorNotBelowThreshold()
    );

    //if collateral isn\'t enabled as collateral by user, it cannot be liquidated
    // This blocks bad debt realization after collateral is fully consumed and flag cleared
    require(
      borrowerConfig.isUsingAsCollateral(collateralReserve.id),
      Errors.CollateralCannotBeLiquidated()
    );
    require(params.totalDebt != 0, Errors.SpecifiedCurrencyNotBorrowedByUser());
  }
```

Synopsis

`validateLiquidationCall` strictly requires the target collateral to be enabled in the user's configuration bitmap. If a user's collateral is fully consumed but bad debt remains, the collateral is disabled, permanently blocking subsequent liquidations and preventing the protocol from realizing the bad debt deficit.

Technical Details

In Aave V3, the bad debt realization flow (`_burnBadDebt`) relies on the `liquidationCall` mechanism to convert unrecoverable user debt into a protocol-level reserve deficit. During a liquidation, if a user's collateral is fully consumed, their `scaledBalanceOf` becomes zero, and the protocol automatically clears the `isUsingAsCollateral` flag for that asset in the user's `UserConfigurationMap` bitmap.

However, `ValidationLogic.validateLiquidationCall` enforces a strict requirement that the `collateralAsset` passed by the liquidator must be actively enabled as collateral by the user (`require(borrowerConfig.isUsingAsCollateral(collateralReserve.id))`).

If a partial liquidation fully consumes the user's last remaining collateral but fails to trigger the `_burnBadDebt` flow in the same transaction (e.g., because the `hasNoCollateralLeft` check evaluates to `false` due to base-currency rounding discrepancies between `totalCollateralInBaseCurrency` and `collateralToLiquidateInBaseCurrency`), the user is left with residual debt and zero enabled collateral. Any subsequent attempt by a keeper bot to trigger the bad debt cleanup via `liquidationCall` will unconditionally revert at the `isUsingAsCollateral` check, because the user no longer has any enabled collateral. This permanently traps the bad debt on the user's balance and prevents the protocol from socializing the loss or covering it via the treasury.

Verification Checklist

1. Verify that `ValidationLogic.validateLiquidationCall` requires `borrowerConfig.isUsingAsCollateral(collateralReserve.id)` to be true for all liquidations.
2. Verify that `LiquidationLogic.executeLiquidationCall` disables the collateral flag if the user's `scaledBalanceOf` reaches zero after the collateral transfer/burn.
3. Verify that the `hasNoCollateralLeft` boolean relies on strict equality between two independently rounded base-currency values, which can evaluate to false even when the actual collateral balance is fully consumed.
4. Verify that `_burnBadDebt` is only triggered if `hasNoCollateralLeft` is true.
5. Verify that if `_burnBadDebt` is missed, there is no alternative entry point to clear the residual debt without passing the `validateLiquidationCall` checks.

Assumptions and Preconditions

- A user's position becomes undercollateralized such that their collateral value is less than their debt value.
- A liquidation is executed that fully consumes the user's remaining collateral balance.
- The `hasNoCollateralLeft` check evaluates to `false` during this liquidation due to base-currency rounding divergence, preventing the immediate execution of `_burnBadDebt`.

Proof of Concept

1. A user holds 1000 USDC as collateral and 1100 DAI in debt. Their health factor is below 1.
2. A liquidator calls `liquidationCall` to liquidate the maximum allowed amount (100% close factor due to HF < 0.95).
3. The liquidation logic calculates that all 1000 USDC must be seized. The user's USDC `scaledBalanceOf` drops to 0.
4. The protocol automatically disables USDC as collateral for the user (`setUsingAsCollateral(..., false)`).
5. Due to a 1 wei rounding difference in base-currency conversions, `vars.totalCollateralInBaseCurrency == vars.collateralToLiquidateInBaseCurrency` evaluates to `false`.
6. `_burnBadDebt` is skipped. The user retains 100 DAI of unbacked debt.
7. Later, a protocol keeper attempts to clean up this bad debt by calling `liquidationCall` again.
8. The call enters `ValidationLogic.validateLiquidationCall`, which checks `borrowerConfig.isUsingAsCollateral(USDC)`.
9. Because the flag was disabled in step 4, the check returns `false`, and the transaction reverts with `CollateralCannotBeLiquidated()`.
10. The 100 DAI bad debt is permanently locked and cannot be moved to the reserve's deficit.

Remediation

Modify `validateLiquidationCall` to bypass the `isUsingAsCollateral` check if the user is in a bad debt scenario and the liquidator is attempting to trigger a bad debt cleanup (e.g., when `debtToCover` is 0 or when the user has no collateral left globally). Alternatively, relax the check to only apply if the user actually has a non-zero collateral balance.

```
require(
borrowerConfig.isUsingAsCollateral(collateralReserve.id) || params.healthFactor == 0,
Errors.CollateralCannotBeLiquidated()
);
```

Status

Fixed at 75145ad021c123a14d9190c67bd30e6867e22cb7

6. Conclusion

We triaged findings from the AI audit and checked the fixes done by the team.

We consider state 75145ad021c123a14d9190c67bd30e6867e22cb7 to be secure against the specific vulnerabilities flagged by the AI auditor.